APSC 200 Project Development

Multi-Agent Differential Drive Robotic System

Project Information

Leo Biancaniello

Will Beveridge

Queen's University

Mathematics and Statistics Department

Summer 2026

Contents

System Overview 3
1 Software 4
1.1 Tracker 4
1.1.1 ArUco Markers 4
1.2 Server 5
2 Firmware 7
2.1 Connection 7
2.2 Motion Planner 7
2.3 Motor Control 8
2.3.1 Motors 8
2.3.2 Encoders 8
2.3.3 Pulse Counter 9
2.3.4 Control Law 9
2.3.5 Spot Turning Control 11
2.4 Battery Management 12
3 Hardware 16
3.1 Chassis 16
3.1.1 3d Print Settings 16
3.2 Assembly 16
Appendix A: Electronics 19
A.1 Component Information 19
A.2 Pin Assignments 21
A.3 Pin Information 22
Appendix B: Testing 23
B.1 Feedforward 23
B.2 Proportional and Integral 24
B.3 Turning 24
Appendix C: Upgrades 24

System Overview

S

The robots operate on a consistent smooth tile floor

Path following (Free) vs Trajectory Generation (Timed)

High and Low level control: Pure Pursuit,

Our goals are Position and Heading, but conventional control is in Linear and Angular Velocity

Collision avoidance should be on the robot itself. If it leaves the frame the server should keep running for the other robots, the one that no longer sees itself in the packets should just stop

The server should send to port 5005, but packets for manual control of individual robots should be sent to port 5006

1 Software

Software constitutes everything ran by the laptop. The system will use a distributed approach where the laptop is responsible for locating the position and heading of each robot, which it then communicates to all the robots that take the information and compute their new headings locally.

“initiation.py” (Should show what the UDP server is sending in the console) (On screen should be (ID, X, Y, Pose) (Simplify start up: List all robot ID’s being used then send 0 when you are done)

“tracker.py”

“udp.py”

Decouple the GUI from the tracking loop so that packets are sent as fast as possible and so that computation for the GUI doesn’t delay the tracker

1.1 Tracker

A **Logitech C922 Pro Stream Webcam** that runs at 30 fps with a **1920x1080p** resolution is used to detect the **ArUco** markers on the robots using **OpenCV’s** computer vision algorithm.

The camera is mounted overhead so there is no lens distortion or optical illusion, thus no anti-warping calibration is needed for the camera. The camera just needs its **focus** to be set using findFocus.py. The current best focus is a value of **15**.

Since an external camera is being used, the camera source in the code needs to be changed from 0 to 1 if the laptop has a webcam*

The camera can view **3600x2030mm** of the floor from a **9ft** ceiling, so the algorithm should apply a cartesian plane to the center of the screen and convert the robot’s pixel positions to **millimetres**.

The robot’s positions are averaged from the detection of their four corners and the robot’s headings are taken as the angle between two of their corners in **radians** and are normalized to $[-\pi, \pi]$ where $0°$ is facing right

DISPLAY FRAME RATE ON CAMERA FEED

1.1.1 ArUco Markers

The markers for the robots are a series of pixels that encode a binary number using black and white cells. The marker patterns are unique under rotation so that the pose of the robot can be determined from its marker orientation.

The algorithm detects the markers by looking for the black to white transition on it’s borders so it is important to always have a white border around the markers. The current markers are **95x95mm** with a **2.5mm** white border to make them **100x100mm** which comes out to **375x375** pixels.

When detecting markers, the code returns the position of its four corners and the id of the marker. Then after detection the pose is estimated with the axes being shown (X-axis is Red, the Y-axis is Green, the Z-axis is Blue).

OpenCV has libraries on ArUco markers that provide dictionaries of different marker sizes and number of markers. It is better to use a shorter dictionary that only includes as many markers as you need as the inter-marker distance in the dictionary affects the cameras noise tolerance. We will use the **4x4_50** dictionary.

The markers are printed on matte 100lb cardstock that they do not bend as easily and have less glare

1.2 Server

A **UDP** (User Datagram Protocol) server will be used to broadcast the robots position data from the tracker to the robots. The UDP server will broadcast all robot positions in one packet to the address 255.255.255.255 on **port 5005** on the router at a rate of **20 Hz** to match the **20 Hz** robot heading logic

BROADCAST_IP = "255.255.255.255"

Since the UDP broadcast goes to every device n the network simultaneously, all robots receive the same packet at the same time.

Packets will be in the form:

- Byte 0 is the Packet Type (uint8) (RUN or STOP)
- Byte 1-4 are Sequence Numbers (uint32)
- Byte 5 is the Number of robots N (uint8)
- Bytes 6+ are the robot records
    - 1 byte for Robot ID (uint8)
    - 2 bytes for X position (int16) - millimeters
    - 2 bytes for Y position (int16) - millimeters
    - 2 bytes for Heading (uint16) - radians x 1000 (scaled for int16 transmission)

Big-Endian format should be used so that byte order is unambiguous between the laptop and ESP, and information should be transmitted in **millimetres** and **radians**.

The server should have a response if a robot that is said to be active is not seen onscreen. This could be broadcasting a specific position (-9999, -9999) to indicate that the robot was lost or it could have a different response to tell the robot to stop.

WHEN THE ROBOT GOES OUT OF FRAME, THE SERVER JUST REPEATEDLY BROADCASTS IT'S LAST POSITION

There should be a start button. So when the server initializes, it should be in the stopped state, then you manually start it up

Binding SERVER_IP = "" doesn't affect the port. The bind tuple (IP, PORT) has two independent parts: the IP picks which interface on your laptop the socket attaches to, and the port is what your robots actually key on.
Your server broadcasts to port 5005, so any robot listening on port 5005 receives the packet — no matter what your laptop's bind IP is. The robots never see your bind IP; they just watch for traffic on the agreed port.
So the only thing that must match is the port: server sends to 5005, robots listen on 5005. Use SERVER_IP = "", keep SERVER_PORT = 5005, and you're set.

WHAT IS THE CAMERA LATENCY?

Test packets getting received

Fresh packets! Fresh position data

Don’t read the same frame object twice

Each camera frame should get processed once

Synchronize main loop to the actual camera

Wait for frame count to change

2 Firmware

IMPORTANT: LEDC API CHANGE: Use New API v3.0+, ledcAttach(pin, freq, res) & ledcWrite(pin, duty) & ledcDetach(pin)

With 20 kHz frequency and 8-bit resolution

The code that runs on the robots decodes the data from the server, determines the robots next headings, then controls the motors to move the robot.

Serial communication should be at a Baud Rate of **115200**

BATTERY MONITORING IS JITTERY AND THE VALUES NEED TO BE SMOOTHED

2.1 Connection

The robots have a built in WiFi module that will be used to read the port on the server to get the robots positions. Upon startup, the robots onboard LED should be RED when they are not connected to WiFi. Then once they connect to the specified network they should turn BLUE. Then when the robots are able to read the packets from the port they should turn GREEN.

The robots should connect to “RobotWifi” with password “12345678” and port 5005 (ALL IT NEEDS IS THE THREE PIECES OF INFO)

There should be a watch dog timer on the robots so that if they do not receive a packet for **1000ms** they stop to avoid driving off screen.

Always use the newest packet in the sequence and discard old ones

2.2 Motion Planner

This should run at **20 Hz**. The robot doesn’t need to make changes every loop, but it should compute everything to be able to make a decision

The algorithm needs to compute what motor speeds are needed to get the robot to it’s next position using the formulas:

$$RPM_L = \frac{\left(v - {}^{L}/_{2} \times \omega\right)}{2\pi r} \times 60$$

$$RPM_R = \frac{\left(v + {}^{L}/_{2} \times \omega\right)}{2\pi r} \times 60$$

v is the desired velocity

ω is the desired angular velocity

r is the radius of the wheel which is **32.5mm**

L is the wheelbase between the two wheels which is **125mm**

2.3 Motor Control

Two motors will spin at different speeds with the same power due to manufacturing tolerances. For the robots to drive straight and turn accurately, we need to control the motor speeds so that the wheels spin at the desired RPM (**60-80 rpm**). Thus, we will use **feedforward** plus **proportional** and **integral** control.

We will run two separate control loops for each motor at **100 Hz**.

NEED TO TEST MIN PWM FOR STRAIGHT AND TURNING

Without battery voltage reference, the minimum duty to spin seems to be 70-75

2.3.1 Motors

The motors used are rated for a 6V input with a speed of 120 rpm at 0.20 kgcm of torque

- IN1 and IN2 control direction. Having IN1 HIGH and IN2 LOW means forward, while the reverse means backwards. Both LOW means coast, and both HIGH means brake
- STBY enables the motor driver so it must be HIGH to operate
- PWM expects a pulse-width modulated square wave where the duty cycle controls the average motor voltage

The PWM channel is configured at 20,000 Hz with 8-bit resolution so it runs a frequency of 20,000 Hz where the Duty Cycle can be set from 0 to 255 where 0 is 0% and 255 is 100%

2.3.2 Encoders

The encoders are Magnetic Wheel encoders with two hall effect sensors that output in quadrature (A/B channels).

The encoders have A and B channels that generate square waves and are shifted so that the leading wave can indicate the direction of the motor spin. Channel A leading channel B means the motor is spinning CCW.

The state of the motor is determined by comparing the current encoder state with the previous encoder state and using a lookup table to determine if there is +1, -1, or 0 ticks.

The formula for the number of ticks per revolution is $7 \times Gear\ Ratio \times Number\ of\ Edges$. So, counting only the rising edge of one channel will give 350 ticks, while counting the change gives 700 ticks, and counting the change on both channels gives **1400 ticks** which we will use.

2.3.3 Pulse Counter

Every time the encoders signal has a rising or falling edge, the CPU needs to take an interrupt to decode the signal. The ESP's hardware **PCNT** (Pulse Counter) peripheral can count encoder pulses directly I hardware, so all the CPU needs to do is read the counter register when it's needed.

The ESP32-S3-DEVKITC-1 has 4 hardware PCNT units (PCNT_UNIT_0 to PCNT_UNIT_3). Unit 0 will be used for the Left motor and Unit 1 will be used for the Right motor. In each unit are two other channels which will be used to read the two channels of each encoder.

Channel 0 is for Encoder Channel A and Channel 1 is for Encoder Channel B with their logic flipped in order to follow proper sequencing such that 00 -> 01 -> 11 -> 10... is Forward and 00 -> 10 -> 11 -> 01 is Backwards.

The ticks are stored in a 16-bit integer that wraps at ± 32767. The program needs to use the current and last reading of ticks as that difference will be how much the wheel spun between intervals. The 16-bit integer arithmetic wraps will be automatically handled by the delta math. Just make sure that currentTicks and deltaTicks are both **int16_t** so that they wrap around the same.

In the function there is an invertDirection clause to make it so that the readings are dependent on if the robot is moving forwards or backwards, so the left unit is inverted and the right unit is normal.

The PCNT can accumulate ticks before the first loop which will through off the delta. Initially reset the accumulation or do something to prevent this issue.

2.3.4 Control Law

In this system, we view the input as the desired RPM from the motor, the control input being the PWM duty that we send, and the output being the actual speed of the motor which we read from the encoder.

Since we know somewhat about the dynamics of the motors, we will measure the PWM duty required to reach different RPM of the motor and then use those values for **Feed forward** control and then additionally use **Proportional** and **Integral** control to get to our desired output.

The manage the **Integral** control, there is an anti-wind up clause and clamping so that the PWM duty never exceeds the safe rating of ~70%

NEEDS TO HANDLE SPINNING IN EITHER DIRECTION

The feedforward model duty = Kv·RPM + Ks is only valid for forward motion. The Ks term represents the static friction offset — the extra duty needed to overcome stiction and start moving. Because friction always opposes motion, this offset must always push *in the direction of travel*.

For a negative (reverse) RPM command, the sign of the Ks term must follow the command direction:

- **Forward (RPM > 0): duty = Kv·RPM + Ks**
- **Reverse (RPM < 0): duty = Kv·RPM − Ks**

Generalized: duty = Kv·RPM + sign(RPM)·Ks

If Ks is left positive for a reverse command, the offset partially cancels the command near zero RPM, making the motor sluggish to break free or unresponsive at low reverse speeds. Note that "inverting Ks" means flipping the *sign of the term* to match direction — Ks itself remains a positive constant.

Companion changes required for reverse to work:

1. **Signed feedforward output — compute the duty magnitude from |RPM|, then apply the command's sign to the result.**
2. **Direction-aware motor drive — the motor function must set the H-bridge direction pins from the sign of the duty, and write the magnitude to the PWM.**
3. **Symmetric saturation — the PI output and saturation limits must span [−MAX_DUTY, +MAX_DUTY] instead of [0, MAX_DUTY], checking both rails.**
4. **Encoder measurement — already handles sign correctly (PCNT counts down for reverse), so no change needed; the error term works in both directions.**

Accuracy note: Forward and reverse Ks are often not equal, due to direction-dependent gearbox and brush friction. For best accuracy, characterize each direction separately to obtain Ks_forward and Ks_reverse. Asymmetric Ks is the likely cause if the robot tracks straight forward but veers in reverse.

Thus the control law is

$$PWM\ duty = \left(K_v \times RPM_{desired} + K_s \times sign(RPM)\right) + \left(K_P \times Error + K_I \times \int Error\right)$$

Where Kv is the duty per RPM and Ks is the minimum PWM needed to overcome static friction which are determined through testing. It seems that K_v should be around 1 and K_s should be around 46.

A good starting point for K_P is 2.5, and for K_I is 1

Acceleration

Startup Acceleration Logic (Slew-Rate Limiting)

Problem. The largest source of heading error is the initial startup. When both motors are commanded from zero to a target RPM instantaneously, they do not break free at the same moment — small differences in static friction (Ks) and gearbox drag mean one wheel starts turning a few milliseconds before the other. During that brief window the robot pivots slightly. Because heading error is the integral of the speed difference between the wheels, any error that occurs before the PI loop catches up is permanent — the loop matches wheel *speeds* thereafter but never undoes the initial pivot.
Solution. Rather than commanding the target RPM instantly, ramp it up gradually from zero using a slew-rate limit. This keeps both motors in the low-speed region longer, where the PI loop is actively correcting, so the instantaneous speed difference stays small throughout startup. A small difference over a gentle ramp produces far less heading error than a large difference over a sharp transient. It also reduces wheel slip and current spikes.
Implementation. The control law itself is unchanged. A "current target" is maintained that chases the "commanded target" at a bounded rate each control tick:
maxDelta = MAX_ACCEL_RPM_PER_SEC × dt
rampedTarget moves toward commandedTarget by at most maxDelta per tick
The ramped target (not the raw command) is then fed into the feedforward and error calculations.
Key requirements:

- **Shared ramp profile.** Both wheels must ramp from a common profile and reach their targets simultaneously. For straight driving, both use the same rate and target. For arcing (unequal wheel speeds), the wheels must ramp *proportionally* — scaling each wheel's rate so the ratio between them stays constant and both arrive together — otherwise the faster-arriving wheel briefly curves the path.
- **Rate tuning.** Start around 300–500 RPM/s and tune on the floor using the Serial Plotter. The two actual-RPM traces should rise together and overlap during the ramp. Choose the fastest rate that still gives clean, matched startup; slower reduces heading error but launches sluggishly.

Assessment. Worth implementing where heading accuracy matters, since it directly targets the identified startup failure mode. It is a small, low-risk addition — a few lines feeding a ramped target into the existing loop, with no change to characterization or the control law — at the cost of a slightly slower launch, which is an acceptable trade for straight-line heading.

2.3.5 Spot Turning Control

To turn on the spot, find the lowest DUTY that the robot needs to spin and then make a proportional plus minimum controller where the robot spins and get's it's heading from the server until it is at the desired angle.

The robot should try to turn in an open loop fashion where it only relies on the encoder ticks and then when it thinks it is in the right spot, it reads the next packet to check it's position and adjust

BRAKE instead of COAST when at desired angle

arc_per_wheel = (|degrees| / 360) × π × wheel_track

ticks = arc_per_wheel / wheel_circumference × ticks_per_rev

Cap Spinning at 120 DUTY

NEED MINIMUM TURNING DUTY + WHEELBASE CALIBRATION

2.4 Battery Management

$$V_{motor} = Duty \times V_{battery}$$

$$Duty = \frac{PWM\ command}{255}$$

Cap Vmotor at 6V so we aren't running too much voltage to the motors

The PWM duty cycle is between 0 and 1, but registered from 0-255

$$PWM_{out} = PWM_{command} \times \frac{V_{nominal}}{V_{battery}}$$

Example

$$PWM_{out} = 128 \times \frac{7.2}{8.4} = 109.7$$

The battery readings are smoothed out with filtering. Moving average ring buffer with 16 samples only. It should update the battery voltage value at a frequency of **5 Hz**

Voltage Divider 100 Kohm and 47 Kohm

Batteries 7.2V nominal, 8.4V fully charged, 6.0V is the lowest it can safely drop

Check everything with a multimeter

2.5 Motion Control Code

Motion control is the high-level layer that decides where the robot should go. It outputs a desired linear velocity v and angular velocity ω, which section 2.2 converts to per-wheel RPM and section 2.3 regulates. It runs at 20 Hz, matching the packet rate — there is no point recomputing geometry faster than the camera reports position.

Two behaviours are implemented: path following, where a robot tracks a stored geometric path, and "follow the leader", where a robot trails another robot without knowing its path. Both share the same underlying pure pursuit steering.

2.5.1 Path Following

2.5.1.1 Path generation

The path is stored as a closed list of evenly spaced waypoints, generated at startup from a list of corner vertices and resampled to a fixed spacing of **50 mm**.

Even spacing matters because the lookahead search accumulates distance point-to-point. Unevenly spaced waypoints make the effective lookahead vary with position on the path.

– *buildFromWaypoints()* takes a vertex list and does the resampling

– The last vertex joins back to the first, closing the loop

– Adding a new shape means writing a vertex list only; nothing downstream changes

2.5.1.2 Pure Pursuit

The path following algorithms use pure pursuit the geometric tracker. It fits a circular arc from the robot's current pose to a goal point placed a fixed distance ahead on the path and drives that arc.

Each cycle, at 20 Hz:

Localise on the path. Advance the path index to the nearest waypoint within a forward-only window of 12 points. Forward-only search keeps progress monotonic — the robot cannot slide backwards along its own path, and on a self-crossing path it cannot jump to the wrong branch.

Select the goal. Walk forward from the current index, accumulating arc length, until the lookahead distance is covered.

Transform to the robot frame. Rotate the goal vector by $-\theta$ to get (x_r, y_r). The bearing to the goal is $\alpha = \text{atan2}(y_r, x_r)$.

Compute curvature. $\kappa = 2 \cdot \sin(\alpha) / L_d$, clamped to a maximum.

Convert to velocities. $\omega = v \cdot \kappa$, then feed v and ω into the section 2.2 wheel formulas.

Lookahead distance is the dominant tuning parameter. Too short and the robot weaves and oscillates; too long and it cuts corners. It also sets the tightest corner the robot can track, which constrains what paths are drivable.

If the goal is more than **90°** off the bow the robot stops and turns in place instead, using the section 2.3.5 spot turn. Hysteresis on this (re-entry at **90°**, exit at **25°**) prevents chattering when the bearing sits near the boundary.

Leader values: lookahead **0.15 m**, cruise **0.15 m/s**, maximum curvature **6.0 m^{-1}** (0.167 m radius). Path-lost threshold **0.45** m triggers a full re-acquire.

2.5.2 Follow the Leader

The follower receives only the leader's current (x, y) from the packet. It does not know the leader's path and does not need to.

2.5.2.1 Breadcrumb trail

Each time the leader's reported position moves more than **40 mm**, a timestamped crumb is appended to a 128-entry ring buffer **(≈5.1 m** of history). Spacing crumbs by distance rather than time means a stationary leader does not flood the buffer.

Steering directly at the leader's current position does not work — it cuts every corner, because the follower tracks the chord rather than the arc and spirals inward. Following the recorded trail reproduces the leader's actual route.

2.5.2.2 Target selection

Walk backwards along the trail from the newest crumb, accumulating distance, until the standoff distance (**0.40 m** of trail length, not straight-line distance) is covered. That crumb is the target.

2.5.2.3 Speed control

v = v_leader + K_gap × (distance to target crumb), capped at **0.20 m/s**

The leader's speed is estimated from crumb timestamps and low-pass filtered. On target, the distance term vanishes and the follower matches the leader exactly, so the gap is self-holding rather than commanded. If no new crumb appears for **500 ms** the estimate decays to zero, so a stopped leader produces a stopped follower rather than a collision. A hard floor at **0.22 m** separation stops the motors regardless of what the control law asks for.

2.5.2.4 Steering

Same pure pursuit as 2.5.1.2, with one change: the distance used in the curvature formula is clamped to a minimum of **0.20 m**.

The target crumb sets the speed, but it is the wrong thing to steer at directly. At steady state the follower sits on that crumb, L_d collapses toward zero, and $\kappa = 2 \cdot \sin(\alpha)/L_d$ explodes into the curvature limit — so the better the tracking, the harder it oversteers. Clamping the steering distance separates the two roles.

Follower values: standoff **0.40 m**, minimum gap **0.22 m**, maximum speed **0.20 m/s**, K_gap **0.35**, maximum curvature **3.0 m^{-1}** (**0.33 m** radius), minimum steering lookahead 0.20 m.

2.5.3 Safety gates

Checked every cycle before any motion is commanded. Any failure calls a stop that writes the motors off, not merely zeroes the setpoints — zeroing setpoints alone leaves the PWM registers holding their last value and the robot keeps driving.

– Link alive — any packet received recently

– Pose fresh — a packet containing this robot with a valid position

– Server RUN — operator has not pressed stop

– Leader fresh — follower only

Pose freshness is the important one and is distinct from link liveness. A live link says nothing about whether the camera can currently see this robot. Without this check a robot whose marker is lost keeps driving on a frozen position indefinitely.

The follower separates stopping from discarding the trail (4 s versus 1.5 s). A brief packet gap means "do not drive on stale data," but the trail is still a valid record of where the leader went. Wiping it on every gap forces a full standoff rebuild and turns a sub-second network hiccup into roughly 3 s of stopped robot.

2.5.4 Geometric Constraints on Paths

Maximum curvature sets a minimum turning radius, $r = 1/\kappa_{max}$. The follower's limit constrains path design, since it must track whatever the leader drives.

For a zigzag of pitch p and swing s, with $k = s/p$:

– Vertex turn angle = $2 \cdot \operatorname{atan}(k)$

– Diagonal length = $p \cdot \sqrt{(1+k^2)}$

– Straight run-in required each side = $r \cdot k$

– Drivable when $p \geq 2 \cdot r \cdot k / \sqrt{(1+k^2)}$

At **r = 0.33 m** and **k = 0.5** the pitch floor is **0.295 m**; the current path uses **0.40 m**.

Note that zigzag path length is $\text{span} \cdot \sqrt{(1+k^2)}$ — it depends on the ratio only. Halving both pitch and swing doubles the zig count but adds essentially no distance. More distance requires a steeper ratio, which sharpens the vertices and eventually breaks drivability.

3 Hardware

The components will be assembled on two breadboards with a power rail in-between and be housed in a 3d printed chassis.

3.1 Chassis

The robot's chassis are **100x100x90mm** with **2.5mm** thick walls so that the interior is **95x95x90mm**. The wheels are placed on the **centerline** of the chassis so that it can turn in place, and skid pads are added to the front and back to aid in balancing.

The Debossed Text should be Gill Sans, Bolded, 40mm Letter Height, Centered, and 0.5mm Deep

(1mm Deep for Roof)

3.1.1 3d Print Settings

1.75mm PETG will be used with a 25% infill and 0.12mm layer height

3.2 Assembly

The assembly of the robots is very important and it should be done properly so that there are not any hardware issues in the future.

First you must solder extensions onto the battery pack so that the Ground and Power wires are both **20cm**. Then you must solder the Power wire to the switch and make the output wire **10cm**. Then you need to extend all of the motor wires with Male DuPont cables so that they can be stuck into the breadboard. Motor wires should be cut at **9cm** and have **1.5cm** stripped for splicing. Then some **22 AWG** wire should be cut to **12cm** and have **0.5cm** stripped for splicing. They should be connected with a V-Shape Twist Splice.

When assembling the breadboard circuits, **22 AWG** wires should be used and there should be **5mm** of exposed wire on each end to snuggly fit into the holes.

DON'T PUSH ON THE BACK OF THE ENCODER WHEN PUTTING IN MOTORS, IT WILL BEND THE MOTOR SHAFT AND BREAK THE MOTOR

The motors are press fit with their wires going towards the front of the chassis and being routed up through the cable holes.

The Battery Pack goes in wire side first and has it’s wires pulled into the cable holder on the left side of the chassis.

The switch should have the smooth side down and be secured with a zip tie.

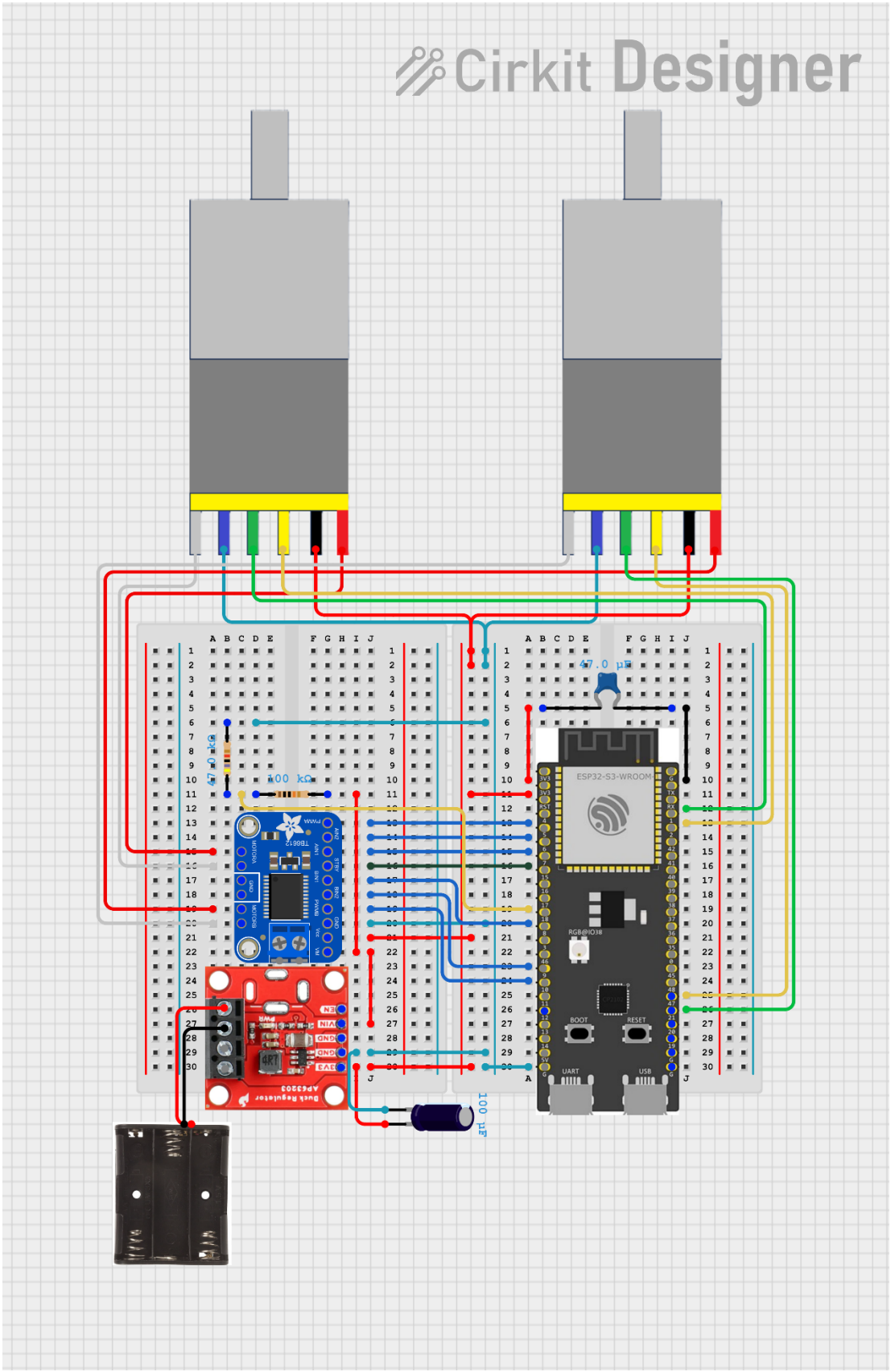


Figure 1: Electronics Schematic

Appendix A: Electronics

The robot uses an ESP32-S3-DEVKITC-1 v1.1 microcontroller that runs at 3.3V logic and has built-in WiFi. The motors have built in encoders and are controlled by a TB6612FNG motor driver. The robot will be powered by six AA batteries with a nominal voltage of 7.2V, going as high as 8.4V when fully charged.

The microcontroller uses Micro-USB and has a USB Port and a USB-to-UART Port. The USB-to-UART port is used for programming and serial monitoring. When programming, the ESP has automatic bootloader entry so the BOOT and RESET buttons do not need to be pressed, but they should be made accessible for troubleshooting.

The motor driver can accept 4.5V to 13.5V, but the motors are only rated for 6V, so the motor driver must be instructed to limit the PWM output to less than 70% to prevent the 7.2V of the battery from damaging the motors.

A voltage divider circuit is connected from the battery to an analog pin on the microcontroller so that the battery voltage can be monitored.

There is a capacitor at the battery terminals and at the microcontrollers terminals to smooth out current flow when components spontaneously demand high currents.

A.1 Component Information

Microcontroller Development Board:

- $23.10 CAD - ESP32-S3-DEVKITC-1-N8R8 v1.1, Adafruit 5336, Datasheet
- 32-bit Dual Core Xtensa LX7 240MHz, 8MB PSRAM, 8MB Flash, Wi-Fi 4
- Can be programmed with exposed BOOT and RESET buttons
- 45 GPIOs (0-21 and 26-48), 4x SPI, 3x UART, 2x I2C, all PWM capable except Powers, Grounds, and Resets
- Has an onboard RGB LED
- Has peak Wi-Fi draws of ~350mA

Motor Driver:

- $10.04 CAD - TB6612FNG DC Motor Driver, Adafruit 2448, Datasheet
- 2.7V – 5V Logic and 4.5V – 13.5V Motor Supply
- 1.2A current per channel with 3.2A peak
- Supports PWM speed control, braking, and direction control
- Internal Flyback, Thermal Shutdown, and Reverse Polarity Protections
- ~90% Efficiency

Motor with built in Encoder:

- $18.06 CAD - N20 DC Motor with Magnetic Encoder 1:50, Adafruit 4638, Datasheet
- 6V DC 0.2W with 100 mA running current and a ~200mA stall current (Can accept 4.5V)

- 200 RPM No Load speed and 120 RPM Rated speed
- 0.20 Kgcm Rated torque and 0.32 Kgcm Stall torque
- 3-5V VCC for the Magnetic Quadrature Encoder with Dual Hall-Effect Sensors
- Encoder output voltage is equal to the encoder supply voltage
- PWM must be limited at 70% to semi-match battery voltage to input voltage*

Wheels:

- $3.00 CAD - 65mm Wheel for N20 Motor, Adafruit 4205, Datasheet
- Hub is specifically designed to match the N20 D-Shaft

Voltage Regulator:

- $10.84 CAD - AP63203 3.3V Buck Regulator, SparkFun COM-18356, Datasheet
- 3.8V – 32V Input
- 2A Max continuous output
- ~88% Efficiency
- Overcurrent, Thermal Shutdown, Undervoltage, Short-Circuit, and Electromagnetic Interference Protection
- Recommends additional external capacitors*

Battery:

- $21.99 CAD - Performance AA Rechargeable Batteries (This is a pack of 12, we need 6)
    - Rechargeable Ni-MH 1.2V 2400mAh
    - 7.2V Nominal and 8.4V when fully charged
- $4.20 CAD - Battery Holder, Datasheet

Voltage Divider:

- R1 = 100KΩ + R2 = 47KΩ
    - Current drain of 61μA
    - Pin voltage never exceeds 3.3V

Capacitors:

- 100μF Electrolytic Capacitor
    - Between battery positive and ground before the converter
    - Must be rated above max battery voltage (8.4V)
    - Positive Leg on positive battery and Negative Leg on ground
- 47μF Ceramic Capacitor
    - Close to ESP power pins
    - One leg on 3.3v ESP pin and other leg on GND ESP pin

Rocker Switch:

- $1.08 CAD - Two Prong Rocker Switch, SparkFun COM-11138, Datasheet
- Backup $4.40 CAD - Two Pring Rocker Switch, Datasheet
    - Two modes: ON and OFF
    - Two pins (One for IN, one for OUT)

2x 400-Pin Breadboards:

- $3.27 CAD - 400 Pin Breadboard
- Needs removable rails

4x 2 Colour LEDS

- $_.__ CAD –
    - 5mm Diameter

A.2 Pin Assignments

Battery:

- Positive terminal goes to VIN on the Voltage Regulator
- Positive terminal goes to PWRIN on the Motor Driver
- Negative terminal goes to GND on the Voltage Regulator

Voltage Regulator:

- GND is tied to GND on the ESP
- 3.3V is tied to 3.3V on the ESP

Left Motor with Encoder:

- M1 (White) is tied to MOTORA2 on the Motor Driver (So AIN1 HIGH, AIN2 LOW is Forward)
- M2 (Red) is tied to MOTORA1 on the Motor Driver
- VCC (Black) is tied to 3.3V on the ESP
- GND (Blue) is tied to GND on the ESP
- C1 (Green) is tied to GPIO1 on the ESP (Direction is handled in code)
- C2 (Yellow) is tied to GPIO2 on the ESP

Right Motor with Encoder:

- M1 (White) is tied to MOTORB1 on the Motor Driver (So BIN1 High, BIN2 LOW is Forward)
- M2 (Red) is tied to MOTORB2 on the Motor Driver
- VCC (Black) is tied to 3.3V on the ESP
- GND (Blue) is tied to GND on the ESP
- C1 (Green) is tied to GPIO21 on the ESP (Direction is handled in code)
- C2 (Yellow) is tied to GPIO47 on the ESP

Motor Driver:

- VCC is tied to 3.3V on the ESP
- GND is tied to GND on the ESP
- PWMA is tied to GPIO4 on the ESP
- AIN2 is tied to GPIO5 on the ESP
- AIN1 is tied to GPIO6 on the ESP
- STBY is tied to GPIO7 on the ESP
- BIN1 is tied to GPIO8 on the ESP

- BIN2 is tied to GPIO9 on the ESP
- PWMB is tied to GPIO10 on the ESP

Voltage Divider:

- Positive Terminal of the battery goes to a 100KΩ resistor then is tied to GPIO18 on the ESP
- The line to GPIO18 has a junction that goes to a 47KΩ that is tied to the negative terminal of the battery

Capacitors:

- 100μF Electrolytic Capacitor: Long (Positive) leg on the positive wire and Short (Negative) leg on the negative wire from the battery pack
- 47μF Ceramic Capacitor: Put as close as possible to the 3.3V and GND pins on the ESP

A.3 Pin Information

- Total Pins:
  - GPIO0-21 and GPIO26-48
- Safe Pins:
  - GPIO1-2, GPIO4-18, GPIO21, GPIO39-42, GPIO47, GPIO48
- Unsafe Pins:
  - GPIO0, GPIO3, GPIO45, GPIO46 are Boot Strapping Pins
  - GPIO19-20 are USB-JTAG Pins
  - GPIO26-32 are Internal SPI Flash / PSRAM Pins
  - GPIO33-37 are Octal SPI Expansion Pins
  - GPIO38 is the Onboard RGB LED
  - GPIO43-44 are TX and RX Pins

Pins 11, 12, 13, 14 are good spots for LED's

Appendix B: Testing

S

B.1 Feedforward

Characterization

Purpose. Determine the feedforward model duty = Kv·RPM + Ks for each motor, where Kv is the duty required per unit RPM and Ks is the static offset (the duty needed to overcome friction and begin moving). This model provides the baseline duty command; the PI loop corrects residual error.
Method. Characterization is performed with the wheels off the ground (no load). No-load conditions give the lowest measurement variance and ensure both motors are measured under identical conditions, which matters because the goal is matching the two motors rather than hitting an absolute RPM. The load-dependent behavior on the floor is handled by the PI loop, not the feedforward.
The routine steps the PWM duty from a minimum to a maximum value, recording steady-state RPM at each step, then fits a line by least-squares regression to extract Kv and Ks.
Measures taken to improve accuracy and consistency:

- **Raised the duty floor** (MIN_MOVE_DUTY = 55) to avoid the low-duty stiction region, where erratic RPM readings distort the fitted offset.
- **Breakaway rejection** — data points below 5 RPM (motor failed to break free) are excluded from the fit.
- **Long, true-timed measurement windows** — 1000 ms per point, timed with micros() rather than assuming the delay duration, to minimize quantization noise.
- **Averaging** — 5 sub-samples averaged per duty point.
- **Bidirectional sweep** — duty is swept up and then down, and both directions are combined in the fit, averaging out backlash and friction hysteresis.
- **Finer duty steps** (CHAR_DUTY_STEP = 10) for more data points and a more stable fit.
- **Warm-up** — motors are run before characterizing so cold-vs-warm gearbox drag does not bias results.
- **Fit quality (R^2)** is computed and reported for each motor, providing a direct measure of how well the linear model fits and flagging noisy data.

Results. After these changes, characterization became consistent and repeatable across runs. Representative fits: both motors converged to Kv ≈ 0.97 and Ks ≈ 31, with $R^2 \geq 0.997$ on every fit. Run-to-run variation in Kv fell to roughly 1.5%, small enough that the PI loop absorbs the difference. This was a marked improvement over early runs, where Ks varied by as much as 10 duty counts between runs and the two motors disagreed by ~8%.

No Load Characterization

No Load PI

Loaded PI

Safe range is 30-120 RPM

B.2 Proportional and Integral

s

B.3 Turning

The value for the wheelbase is what will determine the turning dynamics

Appendix C: Upgrades

S

- Possibly use separate batteries for motors and for logic to prevent sag
- Switch to Lithium Polymer batteries and include a charging circuit on the robot
- Use buck-boost converters for both the Microcontroller and Motor Driver, but add drain protection
- Could switch to better motors that have higher torque at the same speed, but may not be necessary
- Could switch to motors that don't have exposed gears
-
- Implement UWB positioning
- Make the PWM Duty for motor control dependent on the battery level